

Lab 9: Multi-Threaded Bluetooth & LCD Display

Engineer: Stanley Etienne / J00974053

Course: ECEL-360

Date: 04/1/26

Objective

The objective of this lab was to create a multi-threaded program on the ESP32 that uses Bluetooth communication to receive messages from a smartphone and display those messages on an LCD screen. Two separate tasks were created using FreeRTOS: one task handles Bluetooth communication, and the other task updates the LCD display. This lab demonstrates multitasking, wireless communication, and real-time data display using the ESP32.

Materials

- ESP32 Development Board
- 16×2 I²C LCD Display
- Jumper wires
- Breadboard
- Smartphone with BLE app (nRF Connect)
- USB cable
- Arduino IDE

Background and Theory

Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) is a short-range wireless communication protocol used for exchanging small amounts of data efficiently. In this lab, the ESP32 acts as a **BLE server (peripheral)** while the smartphone acts as the **client (central)**. The phone sends messages to the ESP32 by writing to a BLE characteristic, and the ESP32 receives that data.

GATT Structure (Services & Characteristics)

BLE communication is organized using the GATT protocol.

- A **Service** is a group of related data
- A **Characteristic** is a single data value

In this lab:

- A writable characteristic is used to receive messages from the phone
 - A notify characteristic can send data back to the phone
-

LCD Display (I²C Communication)

The 16×2 LCD display uses I²C communication, which reduces wiring to only four connections:

- VCC
- GND
- SDA (data line)
- SCL (clock line)

The ESP32 communicates with the LCD using the LiquidCrystal_I2C library to display text messages.

Multithreading (FreeRTOS)

The ESP32 runs FreeRTOS, which allows multiple tasks (threads) to run at the same time.

In this lab:

- **Task 1 (Bluetooth Task):** Handles incoming messages
- **Task 2 (LCD Task):** Displays messages on the screen

This prevents blocking and allows both operations to run smoothly at the same time.

Wiring Configuration

Component ESP32 Pin Description

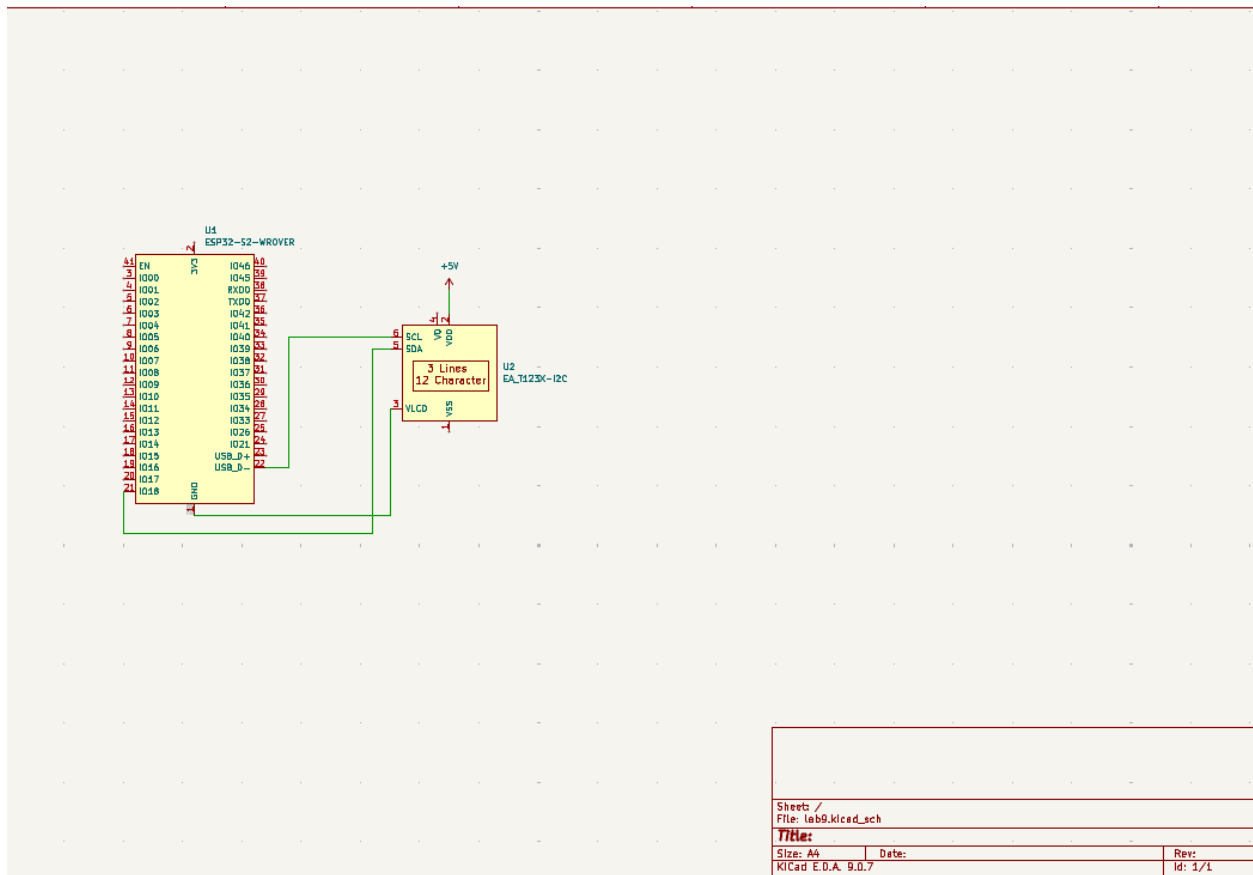
LCD VCC --> VIN (5V) --> Power

LCD GND--> GND--> Ground

LCD SDA--> GPIO21--> I²C Data

LCD SCL--> GPIO22--> I²C Clock

Schematic



Procedure

1. Connected the LCD display to the ESP32 using I²C pins (GPIO21 and GPIO22).
2. Installed required libraries for BLE and LCD communication.

3. Initialized the LCD display using the LiquidCrystal_I2C library.
4. Configured the ESP32 as a BLE server.
5. Created a BLE service and writable characteristic to receive messages.
6. Programmed a callback function to capture incoming BLE data.
7. Created two FreeRTOS tasks:
 - One for Bluetooth communication
 - One for updating the LCD
8. Used queues to safely pass messages between tasks.
9. Uploaded the code to the ESP32.
10. Opened a BLE app on the smartphone and connected to the ESP32.
11. Sent messages from the phone to the ESP32.
12. Observed the messages being displayed on the LCD screen.

Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

// ----- LCD SETUP -----
#define SDA_PIN 21
#define SCL_PIN 22

// Try 0x27 first. If blank, try 0x3F.
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ----- BLE SETUP -----
#define DEVICE_NAME "SE_ESP32_BT_LCD"

// Same UART-style BLE UUIDs used in many ESP32 BLE examples
#define SERVICE_UUID "Levi"
#define CHARACTERISTIC_UUID_RX "umba"
```

```

#define CHARACTERISTIC_UUID_TX "pose"

BLECharacteristic *pTxCharacteristic;
bool deviceConnected = false;

// ----- FREERTOS -----
TaskHandle_t bluetoothTaskHandle = NULL;
TaskHandle_t lcdTaskHandle = NULL;

// Message packet used with queues
typedef struct {
    char text[33];    // 32 chars + null terminator
} MessagePacket;

// Queue 1: BLE callback -> Bluetooth task
QueueHandle_t bleQueue;

// Queue 2: Bluetooth task -> LCD task
QueueHandle_t lcdQueue;

// ----- BLE CALLBACKS -----
class MyServerCallbacks : public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) override {
        deviceConnected = true;
        Serial.println("BLE client connected");
    }

    void onDisconnect(BLEServer* pServer) override {
        deviceConnected = false;
        Serial.println("BLE has disconnected");
        BLEDevice::startAdvertising();
        Serial.println("Advertising restarted");
    }
};

class MyCallbacks : public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) override {
        String rxValue = pCharacteristic->getValue();

        if (rxValue.length() > 0) {
            MessagePacket msg;
            memset(msg.text, 0, sizeof(msg.text));

            // Copy at most 32 chars
            size_t len = rxValue.length();

```

```

        if (len > 32) len = 32;

        for (size_t i = 0; i < len; i++) {
            msg.text[i] = (char)rxValue[i];
        }
        msg.text[len] = '\0';

        // Send to BLE processing task
        xQueueSend(bleQueue, &msg, 0);
    }
}
};

// ----- HELPER FUNCTIONS -----
void displayMessageOnLCD(const char *msg) {
    lcd.clear();

    int len = strlen(msg);

    // If message fits on one line
    if (len <= 16) {
        lcd.setCursor(0, 0);
        lcd.print("Transmitting:");
        lcd.setCursor(0, 1);
        lcd.print(msg);
    }
    else {
        // Split across two lines
        char line1[17];
        char line2[17];

        memset(line1, 0, sizeof(line1));
        memset(line2, 0, sizeof(line2));

        strncpy(line1, msg, 16);
        strncpy(line2, msg + 16, 16);

        lcd.setCursor(0, 0);
        lcd.print(line1);
        lcd.setCursor(0, 1);
        lcd.print(line2);
    }
}

// ----- TASK 1: BLUETOOTH TASK -----

```

```

void TaskBluetooth(void *pvParameters) {
    MessagePacket incomingMsg;

    for (;;) {
        // Wait forever for incoming BLE message
        if (xQueueReceive(bleQueue, &incomingMsg, portMAX_DELAY) == pdTRUE) {
            Serial.print("Received via BLE: ");
            Serial.println(incomingMsg.text);

            // Optional BLE response back to phone
            if (deviceConnected) {
                pTxCharacteristic->setValue(incomingMsg.text);
                pTxCharacteristic->notify();
            }

            // Forward message to LCD task
            xQueueSend(lcdQueue, &incomingMsg, portMAX_DELAY);
        }
    }
}

// ----- TASK 2: LCD TASK -----
void TaskLCD(void *pvParameters) {
    MessagePacket lcdMsg;

    for (;;) {
        // Wait for a message from Bluetooth task
        if (xQueueReceive(lcdQueue, &lcdMsg, portMAX_DELAY) == pdTRUE) {
            Serial.print("Displaying on LCD: ");
            Serial.println(lcdMsg.text);

            displayMessageOnLCD(lcdMsg.text);
        }
    }
}

// ----- SETUP BLE -----
void setupBLE() {
    BLEDevice::init(DEVICE_NAME);

    BLERServer *pServer = BLEDevice::createServer();
    pServer->setCallbacks(new MyServerCallbacks());

    BLEService *pService = pServer->createService(SERVICE_UUID);

```

```

// TX characteristic: ESP32 sends notifications to phone
pTxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_TX,
    BLECharacteristic::PROPERTY_NOTIFY
);
pTxCharacteristic->addDescriptor(new BLE2902());

// RX characteristic: phone writes text to ESP32
BLECharacteristic *pRxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_RX,
    BLECharacteristic::PROPERTY_WRITE
);
pRxCharacteristic->setCallbacks(new MyCallbacks());

pService->start();

BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->start();

Serial.println("BLE server advertising...");
Serial.println("Device name: SE_ESP32_BT_LCD");
}

// ----- SETUP -----
void setup() {
    Serial.begin(115200);

    // LCD
    Wire.begin(SDA_PIN, SCL_PIN);
    lcd.init();
    lcd.backlight();
    lcd.setCursor(0, 0);
    lcd.print("ESP32 BLE LCD");
    lcd.setCursor(0, 1);
    lcd.print("Waiting...");

    // Create queues
    bleQueue = xQueueCreate(5, sizeof(MessagePacket));
    lcdQueue = xQueueCreate(5, sizeof(MessagePacket));

    if (bleQueue == NULL || lcdQueue == NULL) {
        Serial.println("Queue creation failed!");
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Queue Error");
    }
}

```



```

    while (true) {
        delay(1000);
    }
}

// BLE
setupBLE();

// Create tasks on different cores
xTaskCreatePinnedToCore(
    TaskBluetooth,
    "BluetoothTask",
    4096,
    NULL,
    1,
    &bluetoothTaskHandle,
    0
);

xTaskCreatePinnedToCore(
    TaskLCD,
    "LCDTask",
    4096,
    NULL,
    1,
    &lcdTaskHandle,
    1
);

Serial.println("Tasks created successfully");
}

void loop() {
    // Main loop not used
    vTaskDelay(pdMS_TO_TICKS(1000));
}

```

Results

The system operated successfully during testing. The ESP32 was able to advertise itself as a BLE device and accept connections from a smartphone. Once connected, messages sent from the phone were successfully received by the ESP32.

The Bluetooth task processed the incoming data and passed it to the LCD task using a queue. The LCD task then displayed the received message clearly on the screen.

Conclusion

This lab successfully demonstrated how to combine Bluetooth communication, LCD interfacing, and multithreading using the ESP32. The use of FreeRTOS allowed two independent tasks to run concurrently, improving system efficiency and responsiveness.

The ESP32 was able to receive wireless data from a smartphone and display it in real time, showing how embedded systems can interact with external devices. This lab reinforced important concepts such as BLE communication, task scheduling, and I²C display control.